

Sanitizing Inputs to Protect LLMs Against Prompt Injection Attacks

Project Requirements and Specifications

Washington State University

Voiland College of Engineering and Architecture

School of Electrical Engineering and Computer Science

Scalable Algorithms for Data Science LabYour Sponsor



CyberCougs SCADS

Ryan Brombaugh, William Fralia, and Emily West

2/14/2026

TABLE OF CONTENTS

I. Introduction	4
II. SYSTEM REQUIREMENTS SPECIFICATION	4
II.1. USE CASES	4
II.2. FUNCTIONAL REQUIREMENTS	4
<i>II.2.1. Graphical User Interface</i>	4
<i>II.2.2. Detection of Prompt Injection</i>	5
<i>II.2.3. Data Sanitization</i>	5
<i>II.2.4. Prompt Reconstruction</i>	6
II.3. NON-FUNCTIONAL REQUIREMENTS	5
III. SYSTEM EVOLUTION	6
IV. GLOSSARY	6
V. REFERENCES	6

I. Introduction

Prompt injection is among the top ten concerns for large language models and generative artificial intelligence according to the OWASP GenAI Security Project [1]. Our project aims to address this issue by concentrating on mitigating **prompt injection** through data sources. This will culminate into a three-step pipeline where we can test mitigation techniques. These steps include a web-based graphical user interface for input of a prompt and data, a sanitation step to reformat the original prompt and data which aims to prevent **prompt injection** from the data, and finally the construction of a new prompt that will be fed into various large language models (LLMs) for testing purposes. Our project assumes the user is not malicious, and any **prompt injection** attempts would be only present within the data portion of the prompt.

II. System Requirements Specification

II.1. Use Cases

Our team's system will have use cases for any general-purpose testing of **LLM** prompt injection. While this research is done for SCADS, its code will be open-source and could have applications for other stakeholders. Possible scenarios include other researchers interested in prompt injections, or organizations such as companies or the government that utilize **LLMs**.

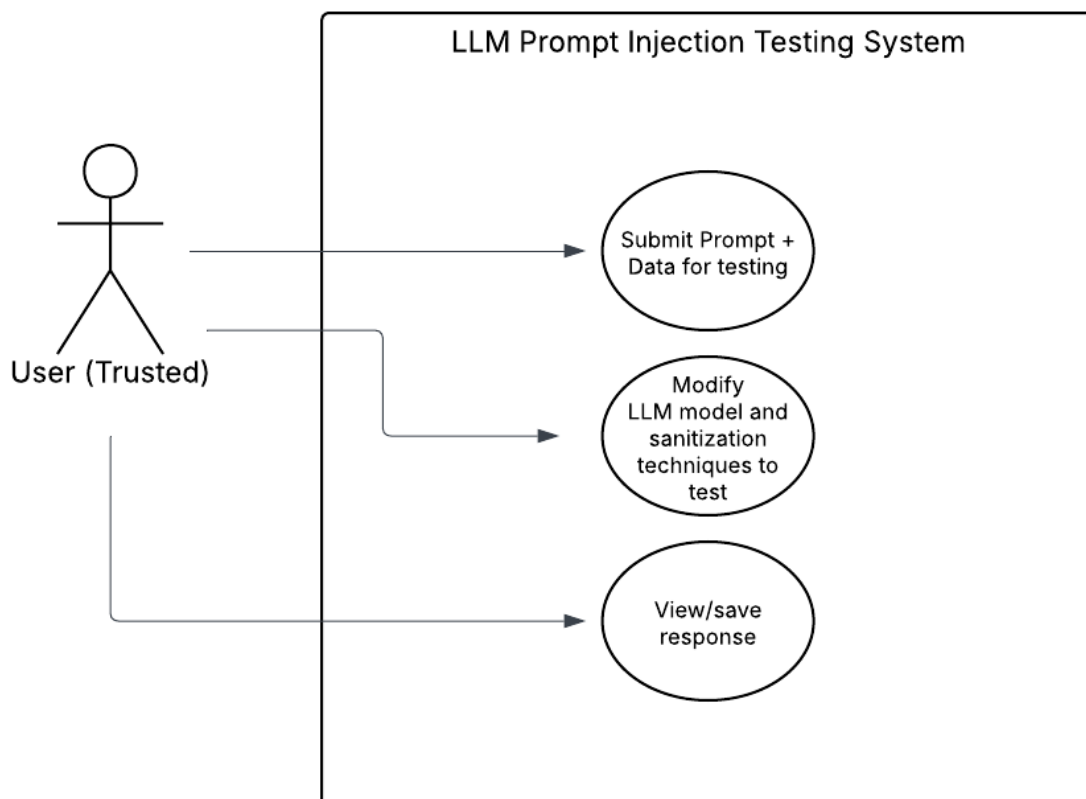


Figure 1 - Use Case Diagram

Use cases for the system will be managed through a web-based user interface. Through the UI, the user will be able to input both a prompt and corresponding data into the system **[FR-1]**. The user will additionally be able to select between different models and sanitization techniques for testing **[FR-2]**. After the system processes the prompt and data, the user will be able to view the produced response **[FR-3]** **[FR-4]**.

II.2. Functional Requirements

Each functional requirement is listed below, organized by the major components of the project. Each functional requirement comes with a description, source of the requirement, and its priority level.

II.2.1. User Interface

[FR-1] Input Separation

Description	The system will provide two inputs to the user in the GUI. One for LLM instruction and one for the data to perform the instruction on. The system will treat instructions and data as separate entities throughout processing, and only modify the data input.
Source	Client (introductory meeting on project scope)
Priority	<u>Priority Level 0</u> : Essential Functionality

[FR-2] LLM Selection

Description	The system will provide the user with a list of available LLMs that can be chosen to query with the sanitized input.
Source	Collaborative between client (second meeting) and team members.
Priority	<u>Priority Level 1</u> : Desirable functionality

[FR-3] LLM Response display

Description	The system will display the LLM response generated from the hardened prompt containing the sanitized data to the user.
Source	Client (introductory meeting on project scope)
Priority	<u>Priority Level 0</u> : Essential Functionality

[FR-4] Malicious data reporting

Description	The system will provide a report on data flagged as malicious. This report could contain statistics on patterns, keywords, and content on detected prompt injections
Source	Team members when discussing project goals.
Priority	<u>Priority Level 2</u> : Extra feature and stretch goal

II.2.2. Detection of Prompt Injection

[FR-5] Injection Pattern Analysis

Description	Detect instruction-like language, role assignment, commands, override attempts, and named entity manipulation in user-provided data.
Source	Client (discussion on project strategy)
Priority	<u>Priority Level 0</u> : Essential Functionality

[FR-7] Secondary LLM screening

Description	As an initial detection screening, pass the user-supplied data to another LLM and ask the LLM if the data contains a prompt injection.
Source	Client (discussion on project strategy)
Priority	<u>Priority Level 1</u> : Desirable functionality

II.2.3. Data Sanitization

[FR-9] Malicious data masking

Description	Apply redaction, removal, modification, or replacement of data portions determined to be prompt injections.
Source	Client (discussion on project strategy)
Priority	<u>Priority Level 0</u> : Essential Functionality

[FR-10] Data integrity preservation

Description	Preserve meaning of benign data and ensure that sanitized data does not override user instructions.
Source	Team members during a meeting with client

Priority	<u>Priority Level 0</u> : Essential Functionality
-----------------	---

II.2.4. Prompt Reconstruction

[FR-11] Prompt Hardening

Description	The system will construct a safe prompt using the original user instructions and the sanitized data. Tell the LLM explicitly to not interpret or follow any instructions in the sanitized data.
Source	Client (introductory meeting on project scope)
Priority	<u>Priority Level 0</u> : Essential Functionality

[FR-12] Call to LLM API

Description	The system will call an LLM using the result of the hardened prompt, and receive its response.
Source	Client (introductory meeting on project scope)
Priority	<u>Priority Level 0</u> : Essential Functionality

II.3. Non-Functional Requirements

Non-Functional Requirement	Description
[NFR-1] Performance	The website will not have high traffic, so minimal hosting resources are required (if hosted). The website should be lightweight and allow less than 2 second load times.
[NFR-2] Scalability	The system's resources required to perform sanitization should scale efficiently with data size and not largely impact the speed of prompt requests.
[NFR-3] Accessibility	The system's user interface shall be made with accessibility in mind, following current web accessibility standards.
[NFR-4] UI/UX	The system's user interface and user interface shall be made simply for the required use cases and should prioritize functionality and ease of use.

[NFR-5] Maintainability	The system's components shall be modular in design to allow easy plug-and-play for new or updated sanitization techniques.
[NFR-6] Security	If hosted publicly, should authenticate users before services can be accessed to prevent malicious LLM prompting.

III. System Evolution

Our system will be built under the assumption that our users are trusted, and the prompt input is safe. This assumption means that if the system were to ever be online, we would need corresponding security features to authenticate users and protect LLM tokens and system integrity. This requirement could possibly mean we decide not to host it online and instead only have the code as an open-source tool for the public.

In addition, **prompt injection** attacks may develop over time, becoming more sophisticated and effective to possibly bypass our detection process or resist our efforts to sanitize. These theoretical more resistant attacks could penetrate our **prompt hardening** measures and influence the **LLM** anyway, reducing the effectiveness of our project.

Our project is fundamentally a middleman between the user and **LLMs**. This means that modifications and development on the **LLM**'s end could possibly end up improving or degrading the performance of our system over time. One particular way this could happen is a change in how **LLMs** process prompts which could allow it sidestep our reconstructed prompt in its efforts to separate instructions and data, removing a possible layer of protection between malicious data and the **LLM**.

Lastly, we will be coding everything in Python, which is a slow programming language that may not scale well. However, we do not believe this is likely to be a problem due to the response to our user depending on the output of their **LLM** of choice. Therefore, we are designating this as a possible but negligible risk to the punctual functioning of our system.

IV. Glossary

Prompt Injection – A cyberattack against **LLMs** in which malicious inputs are disguised as legitimate prompts.

LLM - Large Language Model. A type of machine learning mode that specializes in receiving prompts in natural language. Examples include ChatGPT, Claude, and DeepSeek.

Prompt Hardening – Securing the user's prompt against outside interference.

Data Sanitization – Cleaning user input to prevent accidental or purposeful manipulation of the systems meant to process the prompt.

V. References

- [1] OWASP LLM Project Admin, "OWASP Top 10: LLM & Generative AI Security Risks," OWASP Top 10 for LLM & Generative AI Security, Nov. 18, 2024. <https://genai.owasp.org/>